

Semantic Smoothing for Language Models via Distribution Estimation and Embeddings

Swathi Narashiman and Pranay Mathur
Bachelor Thesis

May 18, 2026



What is Distribution Estimation?

Goal:

- Given finite data (counts), estimate a valid probability distribution
- Works for any context: words, sequences, features, etc.

Formally:

$$P(x \mid c)$$

- x : event (e.g., next token, label, outcome)
- c : context (can be empty, unigram, n-gram, or general features)

Requirements:

- $P(x \mid c) \geq 0$
- $\sum_x P(x \mid c) = 1$



- Assume language generated from a model.
- Try to estimate parameters of that model using given data.
- Eg: Assume our language model is Markovian with degree 1, i.e. the next word depends on the current word.
- Too simple of an assumption, language depends usually on future and past words.
- However, we deal with Bigram Models in this work



Example 1

the cat ---

Next Word	Probability
sat	0.52
slept	0.21
ran	0.09
quantum	0.0001

Example 2

deep learning ---

Next Word	Probability
models	0.34
algorithms	0.25
systems	0.18
banana	0.00001

Key Idea

A good language model assigns high probability to **natural continuations** and low probability to unlikely ones.



- Intuitively, we assign to a bigram its probability of occurrence to be its frequency.
- We call this the empirical estimator (Maximum Likelihood Estimator)

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})} = \frac{C(u, w)}{C(u)}$$

- **Observed Example** "deep learning"

$$P(\text{learning} | \text{deep}) = \frac{120}{300} = 0.40$$

- **Unseen Example** "banana quantum"

$$P(\text{quantum} | \text{banana}) = \frac{0}{45} = 0$$



- An empirical estimator assigns 0 probability to unseen bigrams.
- Redistribute probability mass from seen bigrams to unseen bigrams, making the distribution "smoother"
- **Popular smoothing methods**
 - Add- k Smoothing
 - Jelinek-Mercer Interpolation
 - Kneser-Ney Smoothing



Add Constant

- Assumes each bigram, seen or unseen, occurs k more times than it actually has.
- Ensures that even unseen bigrams are assigned non-zero probabilities.
- Usually, we have $0 < k \leq 1$.

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) + k}{C(w_{i-1}) + kV}$$

- V is the vocabulary size.
- $k = 1$ is called Laplace Smoothing.

Intuition: Adds a uniform “pseudo-count” to all possible bigrams.



- Interpolates between higher-order and lower-order models.
- Helps handle data sparsity by backing off to simpler distributions.
- Uses a mixing parameter $0 \leq \lambda \leq 1$.

$$P(w_i \mid w_{i-1}) = \lambda P_{\text{ML}}(w_i \mid w_{i-1}) + (1 - \lambda) P(w_i)$$

Intuition: Blend a specific (bigram) model with a general (unigram) model to avoid zero probabilities.



- Uses discounting and backs off to a lower-order model.
- Considers how many different contexts a word appears in.
- Performs better than simple interpolation for sparse data.

$$P(w_i | w_{i-1}) = \frac{\max(C(w_{i-1}, w_i) - d, 0)}{C(w_{i-1})} + \lambda(w_{i-1}) P_{\text{cont}}(w_i)$$

$$\lambda(w_{i-1}) = \frac{d \cdot N_1^+(w_{i-1}, *)}{C(w_{i-1})} \quad P_{\text{cont}}(w_i) = \frac{N_1^+(*, w_i)}{N_1^+(*, *)}$$

- $N_1^+(w_{i-1}, *)$: number of unique words that follow w_{i-1} .
- $N_1^+(*, w_i)$: number of unique contexts preceding w_i .
- $N_1^+(*, *)$: total number of unique bigrams.

Intuition: A word is likely if it appears in many different contexts, not just frequently overall.



From N-grams to Modern Language Models

- N-gram models rely on fixed context (e.g., bigrams, trigrams).
- Suffer from data sparsity and limited context window.
- Require explicit smoothing techniques (Kneser-Ney, Good-Turing).
- Modern LLMs (e.g., Transformers) model:
 - Long-range dependencies
 - Rich contextual representations
 - Implicit smoothing via training
- Learn probabilities using neural networks instead of counts.

Key Idea: Move from counting words → learning representations.



Evaluation Metrics

- One among the widely accepted metrics is perplexity.
- It is defined as:

$$\text{Perplexity}(w_1, \dots, w_N) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_1, \dots, w_{i-1}) \right)$$

- The cross-entropy loss measures the average negative log-likelihood of a model

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_1, \dots, w_{i-1})$$

$$\text{Perplexity} = \exp(\mathcal{L}_{\text{CE}})$$

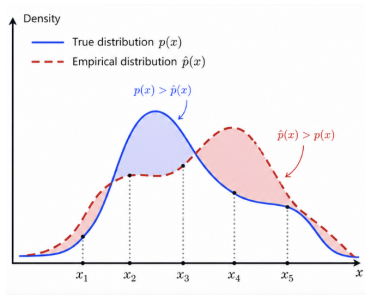
Intuition: Lower perplexity $\implies$ better model (less uncertainty)

Measuring Distribution Difference: KL Divergence

We now have an estimated distribution $\hat{p}$. How close is it to the true distribution p ?

$$D_{\text{KL}}(p \parallel \hat{p}) = \sum_i p(i) \log \frac{p(i)}{\hat{p}(i)}$$

- Measures how much information is lost when using $\hat{p}$ instead of p
- Not symmetric: $D_{\text{KL}}(p \parallel \hat{p}) \neq D_{\text{KL}}(\hat{p} \parallel p)$
= 0 only when distributions are identical



KL divergence intuition



From KL Divergence to Perplexity

- In practice, true distribution p is unknown.
- We approximate using samples $\Rightarrow$ cross-entropy (Information Theory):

$$H(p, \hat{p}) = - \sum_i p(i) \log \hat{p}(i)$$

- Relation to KL divergence: $H(p, \hat{p}) = H(p) + D_{\text{KL}}(p \| \hat{p})$

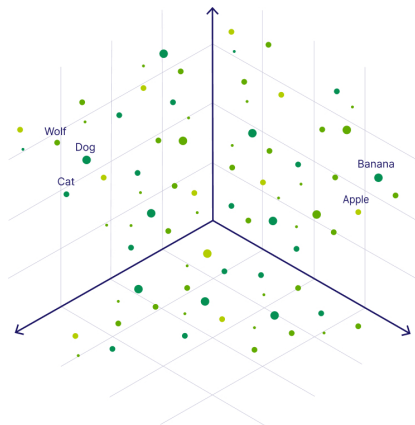
$$\text{PPL} = \exp(H(p, \hat{p})) = \exp(H(p) + D_{\text{KL}}(p \| \hat{p}))$$

Intuition:

$\downarrow$ KL divergence $\Rightarrow$ $\downarrow$ cross-entropy $\Rightarrow$ $\downarrow$ perplexity

Word Embeddings

- Represent words as low dimensional dense vectors.
- Capture semantic and syntactic relationships.
- Popular methods:
 - **Word2Vec** (Mikolov et al., 2013)
 - **GloVe** (Pennington et al., 2014)
- Similar words $\Rightarrow$ similar vectors
- Learned from large corpora using neural models



Example embedding space

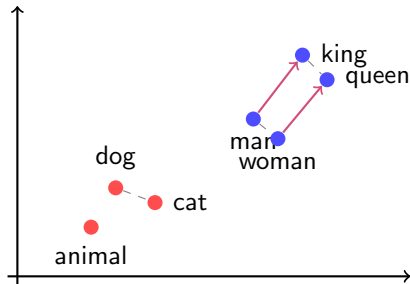
Key Idea: Words are points in a semantic space.

Semantic Similarity

- Similarity between words computed using their embeddings.
- Most common measure: cosine similarity.

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

- Captures semantic relationships:
 - king - man + woman $\approx$ queen
 - paris - france + italy $\approx$ rome



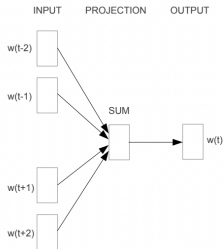
Key Idea: Distance in vector space reflects meaning similarity.

Word2Vec: Learning Word Embeddings

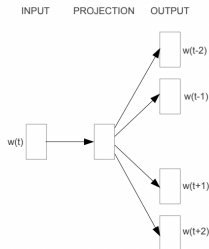
- Proposed by Mikolov et al. (2013)
- Learns dense vector representations:

$$w \mapsto e_w \in \mathbb{R}^d$$

- Two architectures:
 - CBOW
Context $\rightarrow$ Word
 - Skip-Gram
Word $\rightarrow$ Context



CBOW



Skip-gram

Word2Vec architectures



How Word2Vec Trains

Skip-Gram Objective

Given a center word w_t , predict nearby words:

$$\sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j} \mid w_t)$$

Training Pipeline

1. Take a sentence: The cat sat on the mat
2. Choose center word: sat
3. Context words: cat, on
4. Train embeddings so that: $e_{\text{sat}}^T e_{\text{cat}}$ and $e_{\text{sat}}^T e_{\text{on}}$ become large.

Key Insight

Repeated prediction over massive text corpora causes semantically similar words to acquire similar vectors.



CBOW (Continuous Bag of Words)

- CBOW predicts the **center word** using surrounding context words.
- Example: The cat -- on the mat
- Context words: The, cat, on, the
- Predict: sat
- Objective:

$$P(w_t \mid w_{t-m}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+m})$$

- Training process:
 1. Convert context words to embeddings
 2. Average / combine embeddings
 3. Predict center word using softmax
 4. Update embeddings via gradient descent

Observation

CBOW is computationally efficient and works well for frequent words.

GloVe (Global Vectors for Word Representation)



- GloVe learns word embeddings using **global co-occurrence statistics**.

Co-occurrence example:

- "ice" and "steam" may both co-occur with "water" but in different proportions. These ratios encode semantic relationships

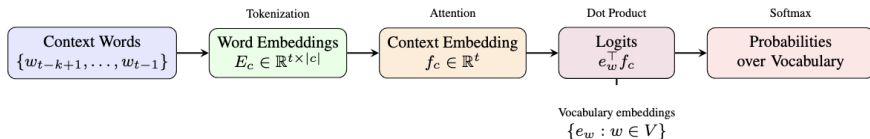
Training objective:

$$J = \sum_{i,j} f(X_{ij}) \left(e_i^\top \tilde{e}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

- b_i , $\tilde{b}_j$ are biases and function $f(X_{ij})$ is a non-decreasing function
- Learns embeddings by factorizing the **co-occurrence matrix**
- Inner product $\approx \log(\text{co-occurrence count})$

Lipschitz-Logit Process in Language Models

- Context words $\rightarrow$ embedding f_c
- Logits via inner product $e_w^\top f_c$
- Softmax gives next-word probabilities



- Small change in embedding $\Rightarrow$ small change in output
- Ensures smooth probability mapping



Lipchitz-Logit Process

- A language model defines next-word probabilities:

$$P(w|c) = \frac{\exp(g_w(f_c) + \epsilon_{c,w})}{\sum_{y \in V} \exp(g_y(f_c) + \epsilon_{c,y})}$$

Components:

- $f_c \in \mathbb{R}^t$: context embedding
- $g_w(\cdot)$: L -Lipschitz logit function
- $|\epsilon_{c,w}| \leq \epsilon$: bounded noise/perturbation

Lipschitz condition:

$$|g_w(f_c) - g_w(f_{\tilde{c}})| \leq L \|f_c - f_{\tilde{c}}\|_2$$



KL Proximity from Embedding Geometry

Key idea:

If contexts c and $\tilde{c}$ have similar embeddings, then their next-word distributions are similar.

Main result:

$$D_{\text{KL}}(P(\cdot|c) \parallel P(\cdot|\tilde{c})) \leq 2L\|f_c - f_{\tilde{c}}\|_2 + 4\epsilon$$

Interpretation:

- Small embedding distance $\Rightarrow$ small KL divergence
- Softmax preserves smoothness of logits
- Embedding geometry controls distribution geometry

Takeaway

Semantic similarity in embedding space induces similarity between next-word probability distributions.



Minimax Theory with Semantic Side Information

Classical result (no side information):

- Minimax KL risk for estimated discrete distribution π from n samples

$$\min_{\pi^*} \max_{\pi} \mathbb{E}[D_{\text{KL}}(\pi \| \pi^*)] \approx \frac{d-1}{2n}$$

Key idea: semantic side information

- Suppose a related distribution $\tilde{\pi}$ is available such that

$$D_{\text{KL}}(\pi \| \tilde{\pi}) \leq \Delta$$

Improved estimator (interpolation):

$$\hat{\pi} = \alpha \pi^*(X^n) + (1 - \alpha) \tilde{\pi}$$

Result:

$$\mathbb{E}[D_{\text{KL}}(\pi \| \hat{\pi})] \leq \min\left(\frac{d-1}{2n}, \Delta\right)$$



Semantic Smoothing for Language Models

- Goal: Improve smoothing in language models using Embedding Similarity
- **Key Insight: Perplexity Decomposition**
 - Log-perplexity decomposes as: ($H(p)$ is fixed)

$$H(p) + \sum_{\text{contexts}} P_{\text{test}}(c) D_{\text{KL}}(P_{\text{test}}(\cdot|c) \| P_{\text{model}}(\cdot|c))$$

Minimize perplexity $\iff$ minimize KL divergence per context

Idea: Smoothing = better distribution estimation after each context.



Semantic Smoothing

- Use semantically similar words as side information.
- For a word w , find synonyms:

$$S_w$$

- Interpolate distributions:

$$P_{interp}(\cdot|w) = \frac{\alpha_w P(\cdot|w) + \sum_{w' \in S_w} \alpha_{w'} P(\cdot|w')}{\alpha_w + \sum_{w' \in S_w} \alpha_{w'}}$$

- Similarity derived from:
 - GloVe
 - Word2Vec
 - GPT-2 embeddings



Experiments Setup

Dataset

- WikiText-2 and WikiText-103 datasets
- Standard train-test split from HuggingFace Datasets

Preprocessing

- Convert all text to lowercase
- Replace:

numbers $\rightarrow$ `<num>` *URLs* $\rightarrow$ `<url>`

- Remove:

punctuation , references , non – ASCII characters

- Normalize whitespaces

Vocabulary

$|V| = 66,753$ (WikiText-103 with 100,000 train sentences)



Embeddings and Tokenization

Embeddings Used

- Word2Vec (300 dimensions)
- GloVe (100 dimensions)
- GPT-2 (768 dimensions)

Tokenization

- Word-level tokenization
- Split sentences using whitespaces
- Add special tokens:

`<bos>` `<eos>`

- GPT-2 tokenizer used when using GPT-2 embeddings

Goal

Use embedding-based semantic similarity as side information to reduce perplexity.



Analysis of the corpus

Goal: Analyze availability of semantic neighbors for rare words (occur once)

Statistic	Value
Total rare words	27390
With embeddings	19922
Avg. neighbors	35.91
Median neighbors	50

Threshold	≥ 0	≥ 1	≥ 2	≥ 5	≥ 10	≥ 20
Rare words	19922	19775	19532	18588	17132	14825

Key observations:

- Most rare words still have many semantic neighbors
- Median neighborhood size is large (50)
- Embedding space is dense even in low-frequency regime

Count-based Semantic Interpolation

- Interpolation constant is given by

$$\alpha_i = \frac{\text{Count}(w_i)}{\sum_{j=1}^m \text{Count}(w_j)}$$

- We apply semantic smoothing to rare words using a threshold.
- Increasing the threshold improves performance

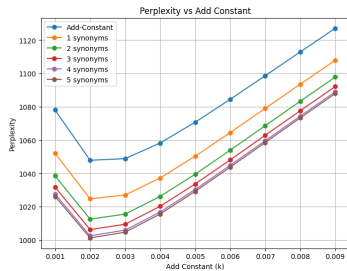


Figure: Threshold = 10

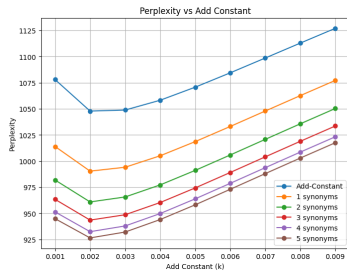


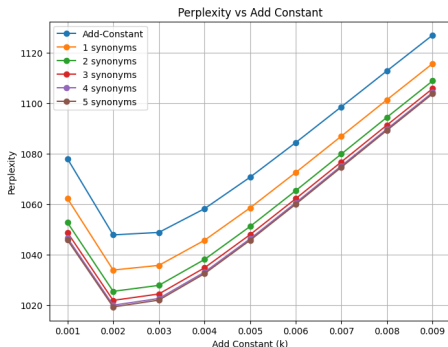
Figure: Threshold = 100

Cosine Similarity Based Semantic Smoothing

- Interpolation constant is given by

$$\alpha_i = \frac{\cos(e_w, e_{w_i})}{\sum_{j=1}^m \cos(e_w, e_{w_j})}$$

- Smoothing across all words
- Improvement in PPL (but not as much as count based method)





Theoretically Motivated Semantic Smoothing

- Empirical results show embedding-based interpolation improves smoothing, but prior methods are heuristic and not derived from KL proximity theory.
- From the KL proximity result:

$$D_{\text{KL}}(P(\cdot|c) \| P(\cdot|c_i)) \leq 2L\|f_c - f_{c_i}\|_2 + 4\epsilon$$

- We define semantic proximity between contexts:

$$\Delta_{cc_i} = 2L\|f_c - f_{c_i}\|_1 + 4\epsilon$$

- Interpolation balances:

$$\frac{d_c}{n_c} \quad (\text{data reliability}) \quad \text{and} \quad \Delta_{cc_i} + \frac{d_{c_i}}{n_{c_i}}$$

Key Idea

contexts with lower KL proximity and more reliable statistics receive higher interpolation weight



Theoretical Smoothing: Results

We evaluate KL-proximity-based semantic smoothing for bigram language models using embedding distances.

- **Semantic proximity:**

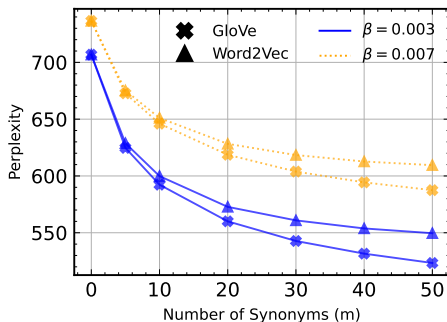
$$\Delta_{w\tilde{w}} = 5\|e_w - e_{\tilde{w}}\|_1$$

- **Neighbor selection (minimax proxy):**

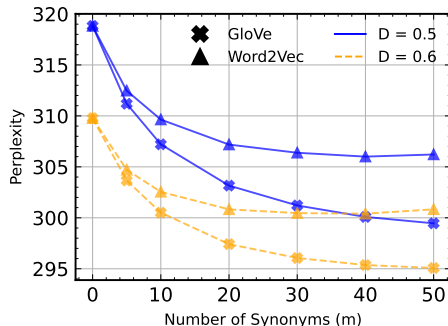
$$\Delta_{w\tilde{w}} + \log\left(1 + \frac{\beta \hat{d}_{\tilde{w}}}{n_{\tilde{w}}}\right)$$

- Top-50 neighbors used, with $\beta = 0.005$, weighting $\phi(x) = 1/x$
- **Results:** Consistent perplexity improvements for both add-constant and Kneser–Ney estimators across GloVe, Word2Vec, and GPT-2 embeddings

Semantic Smoothing: Results



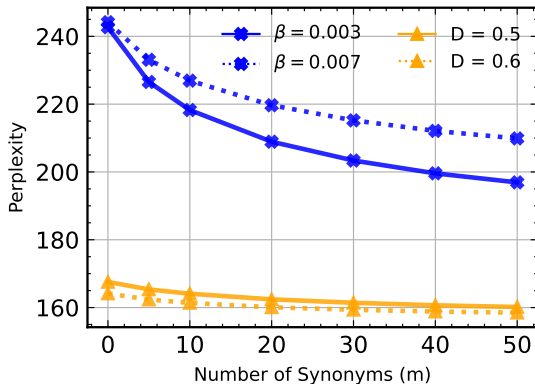
Add-Constant estimator



Kneser-Ney estimator



- Gains persist even under BPE tokenization and strong transformer-based baselines

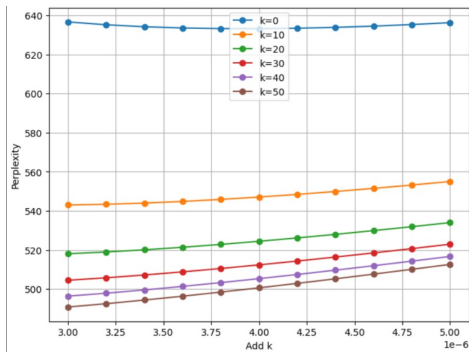


Add-Constant and KN estimator

Add-Count Estimator

- Modification of Add-k was tried, which beat Add-k baselines, and upon interpolation achieved close to Jelinek Mercer performance.

$$\hat{P}_{\text{Add-count}}(w | c) = \frac{N(c, w) + k * (N(c) + k')}{N(c) + k * (N(c) + k') | V|}$$

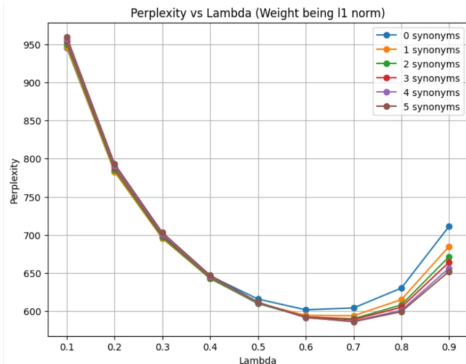


Add Count Estimator

Jelinek Mercer Experiments - Similarity Based



- Similarity based metrics tried out. $\alpha'_w = \frac{1}{\|e_w - e_{w'}\|_1}$, $\alpha_w = 1$ seemed to work well, for reasonable λ values.



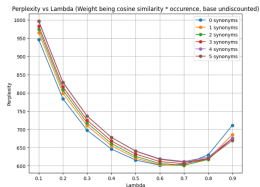
JM Smoothing with l_1



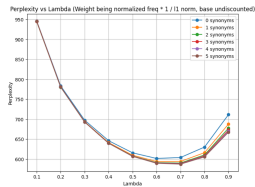
- For the similarity metrics, cosine similarity, $\frac{1}{\|e_w - e_{w'}\|_{\ell_1}}$ and $\frac{1}{\|e_w - e_{w'}\|_{\ell_2}}$ are used, and interpolation is done as:

$$\alpha_{w'} = \frac{\text{Count}(w')}{\sum_{j=1}^m \text{Count}(w_j)} * \text{sim}(w, w') \quad \alpha_w = 1$$

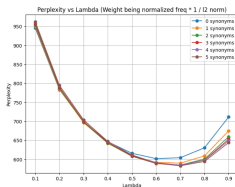
Jelinek-Mercer Experiments - Hybrid



(a) Cosine Similarity



(b) l_1 norm



(c) l_2 norm

Interpolation using counts and similarity metrics

Term-wise Interpolated KN smoothing

- KN decomposition:**

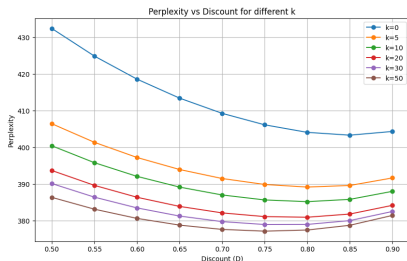
$$P_{KN}^D(w_i|w_{i-1}) = p_1^D(w_i|w_{i-1})$$

$$+ \lambda(w_{i-1}) p_2(w_i)$$

- Semantic interpolation applied to:

$$p_1^D, \lambda(w_{i-1}), p_2(w_i)$$

- Empirical normalization ensures valid probability distribution



Term-wise semantic smoothing improves KN performance (GloVe, Wikitext103)



- **Extension to Longer Contexts**

- Current experiments focus on bigram models.
- Extending the method to higher-order contexts requires:
 - scalable nearest-neighbor search over context embeddings,
 - careful estimation of interpolation weights.

- **Task-Specific Embeddings**

- Current approach uses general-purpose embeddings.
- Training embeddings specifically for semantic smoothing could yield better results.



Thank You